

Security+ 2.3 · SQL Injection

Master CyberSec — Official CompTIA Security+ (SY0-701) Comprehensive Study Guide · **Lesson 2.3 — SQL Injection (5:09)** · [Official YouTube Video ↗](#) · [Professor Messer Study Hub ↗](#)

Executive Lesson Summary: SQL injection (SQLi) occurs when an attacker injects malicious SQL code into input fields, manipulating queries. Examples: 'OR 1=1--' bypasses login; DROP TABLE deletes data; UNION SELECT extracts data. Defenses include prepared statements (parameterized queries), input validation, WAF, and least privilege for database accounts. Never concatenate strings to build queries.

High-Yield Core Concepts Tested on Exam

- SQLi: unsanitized input injected into SQL queries
- Defense #1: prepared statements / parameterized queries
- Defense #2: least privilege (restrict database permissions)
- Other defenses: input validation, WAF, escaping

Exam Tips & Classic Scenarios

- 'input concatenated into a SQL query' = SQL injection
- 'parameterized queries prevent' = defense against SQLi (prepared statements)

Typical Exam Scenario: A login is accepted with input ' OR '1'='1, demonstrating a classic SQL injection attack.

Beyond the Video: Official CompTIA Exam Concepts, Protocols & Traps (Lesson 2.3 — SQL Injection)

Why is this advanced coverage required for SY0-701? Video overviews provide foundational familiarity, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

- **CVSS:** Common Vulnerability Scoring System: Industry standardized 0.0 to 10.0 framework evaluating vulnerability severity (Low, Medium, High, Critical).

Classic CompTIA Traps & Question Framing:

- Avoid rote memorization: CompTIA exam prompts regularly adjust technical constraints requiring you to defend why one valid security control is the *MOST appropriate* or *FIRST step*.
- Scrutinize question qualifiers: 'without additional licensing costs' favors native OS tools; 'granular administrative command accounting' favors TACACS+ over RADIUS; 'non-repudiation' mandates asymmetric private key signing.

Practice Questions with In-Depth Solutions (10 Questions)

Detailed answer key with step-by-step technical rationale and comprehensive distractor breakdown provided below. Attempt answering independently first.

1. SQL command injection in a login field primarily exploits the lack of:

- A)** Disk encryption
- B)** Input validation
- C)** Backup
- D)** Antivirus

2. Inject malicious script into a page to run in the browser of OTHER users:

- A)** XSS (Cross-Site Scripting)
- B)** CSRF
- C)** SQLi
- D)** Buffer overflow

3. In the CVSS 3.1 system, a vulnerability with a score of 9.8 is classified as:

- A) Low**
- B) Medium**
- C) High**
- D) Critical**

4. Corporate scenario (SQL Injection): During a security review and compliance audit aligned with the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'SQLi = input injected into the query without sanitization'. What is the best recommendation or applicable control classification?

- A) Temporarily disable audit logs to accelerate system throughput**
- B) Implement the corresponding solution: SQLi = input injected into the query without sanitization.**
- C) Replace preventive technical controls with informal terms of adherence**
- D) Allow shared access among administrators using generic credentials**

5. Corporate scenario (SQL Injection): During a security review and compliance audit aligned with the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'Defense #1: prepared statement / parameterized query'. What is the best recommendation or applicable control classification?

- A) Implement the corresponding solution: Defense #1: prepared statement / parameterized query...**
- B) Temporarily disable audit logs to accelerate system throughput**
- C) Replace preventive technical controls with informal terms of adherence**
- D) Allow shared access among administrators using generic credentials**

6. Corporate scenario (SQL Injection): During a security review and compliance audit aligned with the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'Defense #2: least privilege (the user's database does not have Drop Table)'. What is the best recommendation or applicable control classification?

- A)** Temporarily disable audit logs to accelerate system throughput
- B)** Replace preventive technical controls with informal terms of adherence
- C)** Implement the corresponding solution: Defense #2: least privilege (the user's database does not have Drop Table)...
- D)** Allow shared access among administrators with generic credentials

7. Corporate scenario (SQL Injection): During a security review and compliance audit aligned with the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'input concatenated into a SQL query' = SQL injection'. What is the best recommendation or classification of applicable control?

- A)** Temporarily disable audit logs to accelerate system throughput
- B)** Implement the corresponding solution: 'input concatenated into a SQL query' = SQL injection...
- C)** Replace preventive technical controls with informal terms of adherence
- D)** Allow shared access among administrators with generic credentials

8. Corporate scenario (SQL Injection): During a security review and compliance audit for the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'parameterized queries prevent' = SQLi defense (prepared statement)'. What is the best recommendation or applicable control classification?

- A)** Temporarily disable audit logs to accelerate system throughput
- B)** Replace preventive technical controls with informal terms of adherence
- C)** Implement the corresponding solution: 'parameterized queries prevent' = SQLi defense (prepared statement)...
- D)** Allow shared access among administrators with generic credentials

9. Corporate scenario (SQL Injection): During a security review and compliance audit aligned with the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'Typical scenario: 'login accepted with input' OR '1'='1' = classic SQL injection.'. What is the best recommendation or applicable control classification?

- A)** Temporarily disable audit logs to accelerate system throughput
- B)** Replace preventive technical controls with informal terms of adherence
- C)** Allow shared access among administrators with generic credentials
- D)** Implement the corresponding solution: Typical scenario: 'login accepted with input' OR '1'='1' = classic SQL injection...

10. Corporate scenario (SQL Injection): During a security review and compliance audit aligned with the CompTIA Security+ exam (SY0-701, Section 2.3), an analyst evaluates the following technical requirement: 'SQLi = input injected into query without sanitization'. What is the best recommendation or applicable control classification?

- A)** Implement the corresponding solution: SQLi = input injected into query without sanitization...
- B)** Temporarily disable audit logs to accelerate system throughput
- C)** Replace preventive technical controls with informal terms of adherence
- D)** Allow shared access among administrators with generic credentials

COMPREHENSIVE ANSWER KEY & DISTRACTOR BREAKDOWN

1) B — Input validation.

✓ **Why it's correct:** The correct alternative is 'Input validation'. SQL injection (SQLi) occurs when an application concatenates user-supplied data directly into a SQL query without proper sanitization or parameterization. In the context of a login field, the attacker inserts special characters (such as single quotes, SQL comments, logical operators) to manipulate the query logic, for example, transforming 'SELECT * FROM users WHERE user='admin' AND pass='password'" into an always-true condition ('OR 1=1 --'). The root cause is the absence of input validation, which should ensure that received data conforms to the expected format (e.g., only alphanumeric characters, maximum length) and, more importantly, the use of parameterized queries (prepared statements) that separate SQL code from data. CompTIA Security+ SY0-701, in the Architecture and Design domain (2.3), emphasizes input validation as a fundamental control to prevent injections. Therefore, the main flaw exploited is the lack of input validation.

✗ **Why the others are wrong:**

- *Disk encryption:* Disk encryption (e.g., BitLocker, LUKS) protects data at rest, ensuring confidentiality if the physical device is stolen. It has no relation to the logic of a SQL query at runtime. Even with fully encrypted disk, SQL injection would still work because the attack occurs at the application layer, not at storage. Therefore, it is a data-at-rest protection technology, not an input validation control.
- *Backup:* Backup is a process of copying data for recovery in case of loss, corruption, or disaster. Although essential for business continuity, it does not prevent SQL injection from being executed. A backup can help restore data after a successful attack, but it does not address the root cause of the vulnerability. SQL injection exploits the lack of input validation, not the absence of backup copies.
- *Antivirus:* Antivirus is software that detects and removes known malicious code, such as viruses, worms, and trojans, based on signatures or heuristics. It does not analyze incoming traffic to a web application for malicious SQL commands. SQL injection is an application attack that can be executed without any malware on the server or client. Therefore, antivirus is not an effective control against SQL injection; the correct defense is input validation and parameterized queries.

2) A — XSS (Cross-Site Scripting).

✓ **Why it is correct:** XSS (Cross-Site Scripting) is the vulnerability that allows the injection of malicious scripts (usually JavaScript) into web pages viewed by other users. In XSS, the attacker exploits the lack of input sanitization and/or output encoding so that the victim's browser executes the code in the context of the vulnerable application. There are three main types: stored (persistent), reflected (non-persistent), and DOM-based. The impact includes session cookie theft, session hijacking, malicious redirection, keylogging, and page defacement. The question describes exactly this scenario: 'Inject malicious script into a page to run in the browser of OTHER users', which is the precise definition of XSS according to SY0-701 objective 1.3 (application vulnerabilities) and 2.2 (application-based threats).

✗ **Why the others are wrong:**

- *CSRF:* CSRF (Cross-Site Request Forgery) is an attack that forces the victim's browser, already authenticated on a site, to perform unintended actions on that site (e.g., bank transfer, password change). The attacker does not inject script into the page; they trick the victim into having their browser send a forged request, leveraging the trust the site has in the victim's session cookie. Therefore, CSRF does not execute code in other users' browsers; it exploits the application's trust in the victim's browser. The question explicitly asks for script injection that runs in other users'

browsers, which is not the CSRF mechanism.

- **SQLi:** SQL Injection (SQLi) is a technique of injecting malicious SQL commands into application inputs to manipulate the underlying database. The attacker can extract, modify, or delete data, bypass authentication, execute operating system commands, among others. The target is the database server, not other users' browsers. SQLi does not inject scripts that run in the victim's browser; it exploits the persistence layer. The question's description mentions 'run in the browser of OTHER users', which is characteristic of XSS, not SQLi.
- **Buffer overflow:** Buffer overflow is a memory corruption vulnerability where a program writes more data than the allocated buffer can hold, overwriting adjacent memory areas. This can lead to arbitrary code execution, system crash, or privilege escalation. The attack occurs in the vulnerable software process, not in other users' browsers. Although browsers can suffer buffer overflow, the described scenario is script injection into a web page for execution in third-party browsers, which is the specific domain of XSS. Buffer overflow is not a technique for injecting scripts into web pages.

3) D — Critical.

✓ **Why it's correct:** In the CVSS 3.1 (Common Vulnerability Scoring System), the base score ranges from 0.0 to 10.0 and is categorized into four ranges: Low (0.1–3.9), Medium (4.0–6.9), High (7.0–8.9), and Critical (9.0–10.0). A vulnerability with a score of 9.8 therefore falls into the Critical range, indicating severe impact and trivial or near-trivial exploitability, with the potential to compromise confidentiality, integrity, and availability massively. CompTIA Security+ requires professionals to interpret these ranges to prioritize remediation.

✗ **Why the others are wrong:**

- **Low:** The Low classification in CVSS 3.1 covers scores from 0.1 to 3.9. Vulnerabilities in this range typically require very specific conditions for exploitation, have limited impact, or require highly privileged local access. A score of 9.8 is far above this range, indicating extreme, non-trivial risk. Associating 9.8 with Low demonstrates a complete misunderstanding of the scale.
- **Medium:** The Medium range in CVSS 3.1 goes from 4.0 to 6.9. Medium vulnerabilities can cause moderate damage, often requiring user interaction or partial exploitation conditions. A score of 9.8 far exceeds this threshold, placing it at the top of the scale. Classifying it as Medium would severely underestimate the urgency of remediation and contradict the standard's own numerical definition.
- **High:** The High range in CVSS 3.1 comprises scores from 7.0 to 8.9. Although it is a serious classification, it is still below the Critical threshold. A 9.8 vulnerability is considered Critical because it typically combines remote attack vectors, low complexity, no required privileges, and high impact on all three security pillars. Therefore, labeling it as High is technically incorrect according to the official CVSS 3.1 specification.

4) B — Implement the corresponding solution: SQLi = input injected into the query without sanitization..

✓ **Why it is correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): SQLi = input injected into the query without sanitization. SQL injection occurs when untrusted user input is concatenated directly into SQL queries without proper validation or parameterization, allowing attackers to manipulate the query and execute arbitrary database commands. Mitigation includes input validation, parameterized queries, and least privilege.

✗ **Why the others are wrong:**

- **Temporarily disable audit logs to accelerate system throughput:** Directly violates the Accounting pillar and forensic traceability. Without logs, the team is blind to incident detection and breaks

compliance standards (PCI DSS, ISO 27001).

- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls that rely solely on user behavior and do not block automated attacks, injections, or runtime malware.
- *Allow shared access among administrators using generic credentials:* Destroys individual accountability and non-repudiation. It makes it impossible to prove who performed critical changes or malicious commands.

5) A — Implement the corresponding solution: Defense #1: prepared statement / parameterized query....

✓ **Why it is correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): Defense #1: prepared statement / parameterized query. Prepared statements with parameterized queries ensure that SQL code and data are separated, preventing attackers from injecting malicious SQL. This is the primary defense against SQL injection. The appropriate action is to implement the corresponding solution.

✗ **Why the others are wrong:**

- *Temporarily disable audit logs to accelerate system throughput:* Directly violates the Accounting pillar and forensic traceability. Without logs, the team is blind to detecting incidents and breaks compliance standards (PCI DSS, ISO 27001).
- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls that rely solely on user behavior and do not block automated attacks, injections, or malware at runtime.
- *Allow shared access among administrators using generic credentials:* Destroys individual accountability and non-repudiation. It makes it impossible to prove who performed critical changes or malicious commands.

6) C — Implement the corresponding solution: Defense #2: least privilege (the user's database does not have Drop Table)....

✓ **Why it's correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): Defense #2: least privilege (the user's database does not have Drop Table).

✗ **Why the others are wrong:**

- *Temporarily disable audit logs to accelerate system throughput:* Directly violates the Accounting pillar and forensic traceability. Without logs, the team is blind to detecting incidents and breaks compliance standards (PCI DSS, ISO 27001).
- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls, rely solely on user conduct, and do not block automated attacks, injections, or malware at runtime.
- *Allow shared access among administrators with generic credentials:* Destroys individual accountability and non-repudiation. Makes it impossible to prove who performed critical changes or malicious commands.

7) B — Implement the corresponding solution: 'input concatenated into a SQL query' = SQL injection....

✓ **Why it's correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): 'input concatenated into a SQL query' = SQL injection.

✗ **Why the others are wrong:**

- *Temporarily disable audit logs to accelerate system throughput:* Directly violates the Accounting

pillar and forensic traceability. Without logs, the team is blind to detect incidents and breaks compliance standards (PCI DSS, ISO 27001).

- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls, rely solely on user conduct, and do not block automated attacks, injections, or malware at runtime.
- *Allow shared access among administrators with generic credentials:* Destroys individual accountability and non-repudiation. Makes it impossible to prove who performed critical changes or malicious commands.

8) C — Implement the corresponding solution: 'parameterized queries prevent' = SQLi defense (prepared statement)....

✓ **Why it's correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): 'parameterized queries prevent' = SQLi defense (prepared statement).

✗ **Why the others are wrong:**

- *Temporarily disable audit logs to accelerate system throughput:* This directly violates the Accounting pillar and forensic traceability. Without logs, the team is blind to detecting incidents and breaks compliance standards (PCI DSS, ISO 27001).
- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls, rely solely on user behavior, and do not block automated attacks, injections, or malware at runtime.
- *Allow shared access among administrators with generic credentials:* This destroys individual accountability and non-repudiation. It makes it impossible to prove who made critical changes or malicious commands.

9) D — Implement the corresponding solution: Typical scenario: 'login accepted with input' OR '1'='1' = classic SQL injection....

✓ **Why it's correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): Typical scenario: 'login accepted with input' OR '1'='1' = classic SQL injection.

✗ **Why the others are wrong:**

- *Temporarily disable audit logs to accelerate system throughput:* Directly violates the Accounting pillar and forensic traceability. Without logs, the team is blind to detecting incidents and breaks compliance standards (PCI DSS, ISO 27001).
- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls, rely solely on user conduct, and do not block automated attacks, injections, or malware at runtime.
- *Allow shared access among administrators with generic credentials:* Destroys individual accountability and non-repudiation. Makes it impossible to prove who performed critical changes or malicious commands.

10) A — Implement the corresponding solution: SQLi = input injected into query without sanitization....

✓ **Why it's correct:** According to the official CompTIA Security+ (SY0-701) exam objectives for lesson 2.3 (SQL Injection): SQLi = input injected into query without sanitization.

✗ **Why the others are wrong:**

- *Temporarily disable audit logs to accelerate system throughput:* Directly violates the Accounting pillar and forensic traceability. Without logs, the team is blind to detecting incidents and breaks compliance standards (PCI DSS, ISO 27001).

- *Replace preventive technical controls with informal terms of adherence:* Informal terms are administrative controls, rely solely on user conduct, and do not block automated attacks, injections, or malware at runtime.
- *Allow shared access among administrators with generic credentials:* Destroys individual accountability and non-repudiation. Makes it impossible to prove who performed critical changes or malicious commands.